Relatório 27 - Prática: Modelos Generativos (III)

Igor Carvalho Marchi

1. Introdução

Este card abordou os modelos generativos, que aprendem a distribuição dos dados para criar exemplos novos em vez de apenas classificá-los. Foram vistas três famílias: os autoencoders variacionais (VAEs) e as redes adversariais generativas (GANs), aplicados a imagens do Fashion MNIST e o GPT, que gera texto prevendo o próximo caractere.

2. Desenvolvimento

2.1 AutoEncoders e VAEs

Um autoencoder comprime a imagem e depois tenta reconstruí-la. O encoder reduz a entrada a poucos números, o chamado espaço latente, e o decoder reconstrói a imagem a partir deles. Para reconstruir bem, o modelo precisa guardar o que realmente define a imagem, como o formato de uma calça ou de um tênis.

No VAE o encoder não devolve um ponto fixo, e sim uma distribuição: uma média e uma variância para cada dimensão do latente. O ponto z é sorteado dessa distribuição. Como o sorteio não é derivável, usa-se o truque de reparametrização, Z = μ + σ x (E), com (E) vindo de uma normal padrão. Assim o gradiente passa por μ e σ normalmente.

O VAE sorteia z da distribuição aprendida e reconstrói a imagem

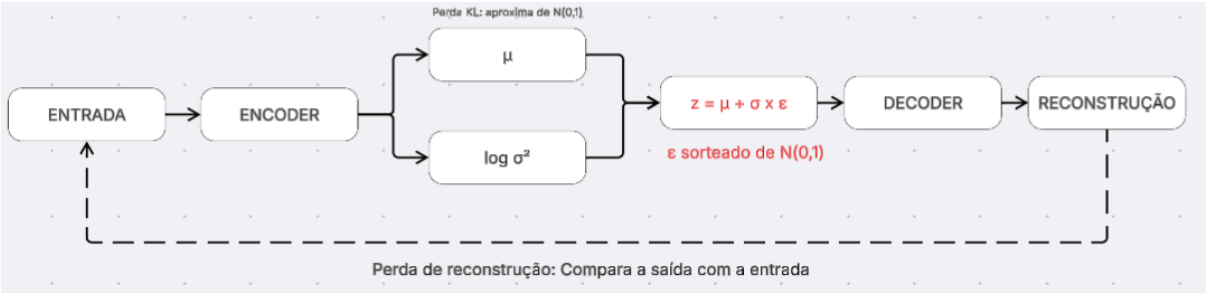


Figura 1 - Funcionamento de um VAE

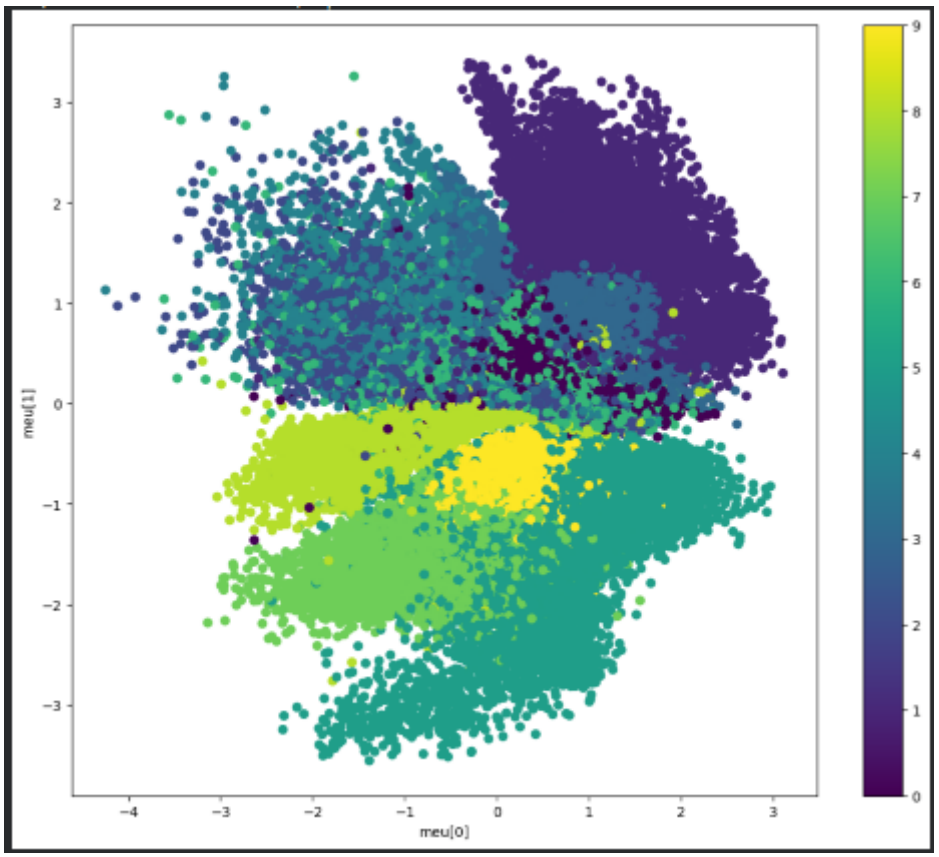


Figura 2 - Médias do espaço latente das 70 mil imagens, coloridas por classe.

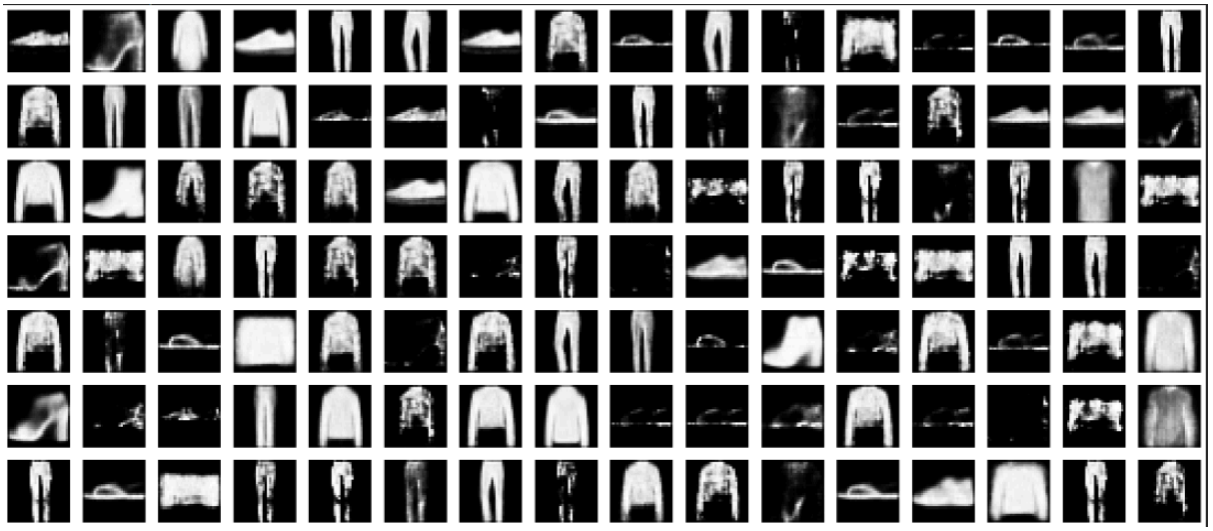


Figura 3 - 256 imagens geradas pelo decoder a partir de pontos sorteados do latente.

O treino combina duas perdas:

- Reconstrução (entropia cruzada binária por pixel): mede o quanto a imagem reconstruída se parece com a original.
- Divergência KL: aproxima a distribuição do latente de uma normal N(0, 1). Isso organiza o espaço latente e permite gerar imagens novas sorteando pontos dele.

No notebook da aula, o VAE foi treinado no Fashion MNIST (70 mil imagens 28x28 de roupas, em 10 classes) com latente de 2 dimensões e peso 3 na KL, o que na prática o torna um BETA-VAE. Em 32 épocas, a perda total caiu de 314,5 para 266,0. O latente de 2 dimensões permite desenhar todas as imagens num plano: as classes formam regiões separadas mesmo sem o modelo ter visto os rótulos. A desvantagem é que as imagens geradas saem borradas, porque 2 números guardam pouca informação.

2.2 GANs

Uma GAN (Generative Adversarial Network) treina duas redes que competem. O gerador recebe um vetor de ruído aleatório e tenta transformá-lo numa imagem realista. O discriminador recebe imagens reais e imagens do gerador e tenta dizer quais são falsas. O gerador melhora quando consegue enganar o discriminador, e o discriminador melhora quando consegue pegá-lo. O equilíbrio ideal, chamado equilíbrio de Nash, é quando o discriminador não consegue mais diferenciar as duas.

O gerador tenta enganar o discriminador, que separa a real de falsa

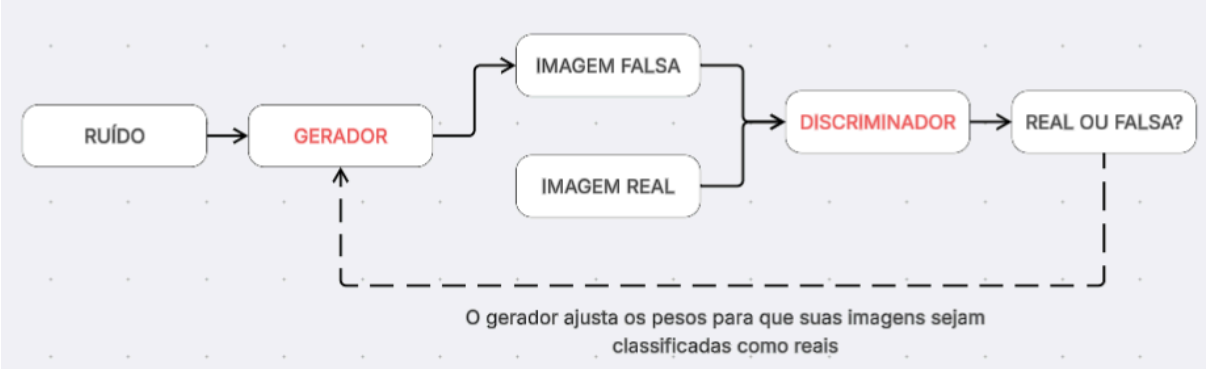


Figura 4 - Funcionamento de uma GAN

No notebook da aula, o gerador transforma um ruído de 150 dimensões em uma imagem 28x28 com camadas Conv2DTranspose, e o discriminador usa camadas Conv2D para dar a probabilidade de a imagem ser real. Em vez do model.fit, cada lote treina primeiro o discriminador e depois o gerador, cada um com seu otimizador. As taxas de aprendizado são baixas e diferentes (1e-5 no gerador e 3e-5 no discriminador), porque se um lado ficar forte demais o outro para de aprender.

Em 30 épocas, o ruído virou roupas reconhecíveis, mas o discriminador continuou dominando: terminou acertando cerca de 80% das imagens, enquanto o gerador o

enganou em apenas 4% das vezes. Também dá para ver o mode collapse: muitas amostras saem parecidas entre si, principalmente camisetas e calças, em vez de cobrir as 10 classes.

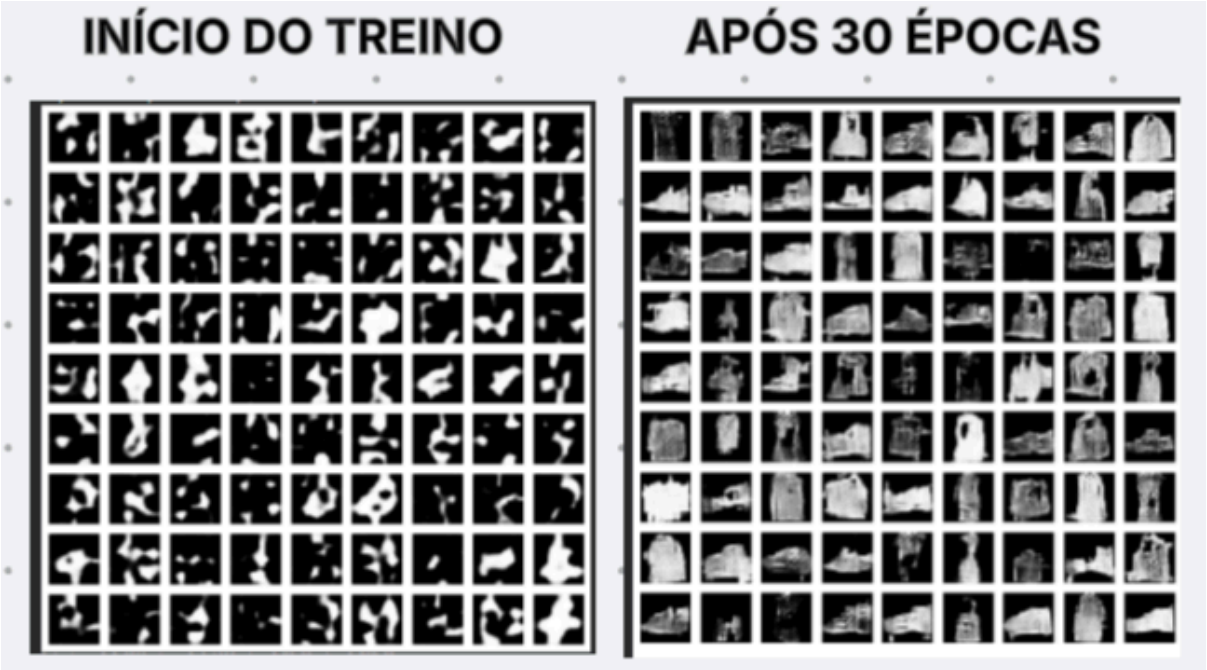


Figura 5 - Saídas do gerador para ruídos aleatórios: manchas no início do treino e roupas reconhecíveis depois de 30 épocas.

2.3 GPT e self-attention

O GPT é um modelo de linguagem: dado um trecho de texto, ele calcula a probabilidade de cada próximo token e sorteia um deles. Por isso não é determinístico: o mesmo início pode gerar textos diferentes. Na aula, baseada no artigo Attention is All You Need, o modelo foi construído do zero em PyTorch, em nível de caractere, com o Tiny Shakespeare (1,1 milhão de caracteres e vocabulário de 65 símbolos).

O texto é convertido em números (cada caractere vira um índice) e dividido em 90% para treino e 10% para validação. Cada exemplo de treino é um trecho de block_size caracteres, e o alvo é o mesmo trecho deslocado em uma posição. Assim, um único trecho ensina o modelo a prever o próximo caractere em todas as posições.

O primeiro modelo foi um bigrama: uma tabela 65 x 65 em que cada caractere lê diretamente as chances do próximo. Ele aprende combinações de letras, mas não enxerga nada além do caractere atual, então o texto gerado não forma palavras.

Para usar o contexto, entra a self-attention. Cada token gera três vetores a partir do seu embedding:

- Query: o que o token procura.
- Key: o que o token oferece.
- Value: a informação que ele passa adiante.

O produto entre a query de um token e as keys dos anteriores dá a afinidade entre eles. Depois do softmax, essas afinidades viram pesos que somam 1, e a saída é a média ponderada dos values:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \times K^T / \sqrt{d_k}) \times V$$

A divisão pela raiz do tamanho da cabeça mantém a variância perto de 1. Sem ela, o softmax fica "pontudo" e cada token presta atenção em um único outro. A máscara causal, uma matriz triangular inferior, coloca menos infinito nas posições futuras, para o modelo não "colar" vendo o caractere que deveria prever.

O modelo completo junta várias cabeças de atenção em paralelo, cada uma aprendendo um tipo de relação, seguidas de uma rede feed-forward aplicada a cada token. Esse par forma um bloco transformer, com conexões residuais e LayerNorm para o treino ficar estável em redes profundas, e dropout contra overfitting. Os blocos são empilhados, e uma camada final transforma o resultado nas chances de cada caractere.

3. Atividade prática

Usei a mesma arquitetura do gpt.py da aula, mas só com notícias da Folha de S.Paulo, em duas etapas: pré-treino com notícias de todas as seções menos esporte, e fine-tuning só com esporte. Deixar esporte fora do pré-treino garante que o fine-tuning ensine algo novo. Para medir o efeito do pré-treino, treinei também um modelo igual do zero, só com esporte, pelo mesmo número de passos.

Limpeza dos dados (limpeza_dados.ipynb). As notícias vieram da base do Kaggle com cerca de 167 mil matérias. Removi as vazias e duplicadas e apliquei a limpeza com regex do card de NLP, que trata a coluna inteira de uma vez, em vez de um laço notícia por notícia. Todos os caracteres foram normalizados para um

vocabulário fixo de 113 símbolos, e cada notícia foi gravada como [seção] Título seguido do texto.

Treino (treino_gpt.ipynb): em relação ao código da aula, acrescentei uma função de treino usada nas três etapas, decaimento cosseno na taxa de aprendizado, precisão mista para acelerar o treino e um checkpoint salvo no Drive a cada avaliação.

4. Conclusão

Os três modelos do card geram dados novos de formas diferentes. O VAE aprende um espaço latente organizado e estável de treinar, mas produz imagens borradas. A GAN gera imagens mais nítidas, mas o treino é um equilíbrio delicado entre duas redes, e no notebook o discriminador dominou do começo ao fim. O GPT, base dos LLMs usados hoje em assistentes e chatbots, mostrou que prever o próximo caractere com self-attention já basta para imitar a estrutura de um texto.

REFERÊNCIAS

Dataset Kaggle - Kaggle

LINK Dos Insights - Lucidspark

Vídeos do Drive - LINK DRIVE

Vídeo do Youtube - https://www.youtube.com/watch?v=kCc8FmEb1nY